

Task 1:

Question 1:

Explanation:

- This question requires loading the October 2024 dataset into a single Spark DataFrame. The Spark API used to achieve this is `spark.read.json()`.
- The `to_timestamp` was used to convert the `created_at` field to a Spark timestamp. This then allows us to obtain the date, hour and dow(day of week) metrics using the Spark functions: `date_format`, `hour` and `dayofweek`
- To cache the DataFrame the operation `cache()` is used which helps avoid repeated actions such as computing values that were already computed, hence improving efficiency
- The `show()` operation was used to display the 10-row preview which ensures the data is being loaded and processed correctly
- As there was a large volume of datasets to be processed the run time was very long, thus each part was tested on a sample before being used on the full dataset

Schema:

The `printSchema` API was used in the SPARK script to display the schema

The schema contained many fields and so I was unable to screenshot the whole thing into this report but from my screenshots below you can see the root and some main top-level fields. You can also see the end of the schema which contains the fields that were derived(`created_at_ts`, `date`, `hour` and `dow`).

```
2025-12-18 15:08:27,335 INFO codegen.CodeGenerator: Code generated in 12.062883 ms
2025-12-18 15:08:27,389 INFO spark.SparkContext: Starting job: csv at NativeMethodAccessorImpl.java:0
2025-12-18 15:08:27,393 INFO scheduler.DAGScheduler: Registering RDD 35 (csv at NativeMethodAccessorImpl.java:0) as input to shuffle 2
2025-12-18 15:08:27,394 INFO scheduler.DAGScheduler: Got job 9 (csv at NativeMethodAccessorImpl.java:0) with 1 output partitions
2025-12-18 15:08:27,394 INFO scheduler.DAGScheduler: Final stage: ResultStage 15 (csv at NativeMethodAccessorImpl.java:0)
2025-12-18 15:08:27,394 INFO scheduler.DAGScheduler: Parents of final stage: List(ShuffleMapStage 14)
2025-12-18 15:08:27,394 INFO scheduler.DAGScheduler: Missing parents: List(ShuffleMapStage 14)
2025-12-18 15:08:27,396 INFO scheduler.DAGScheduler: Submitting ShuffleMapStage 14 (MapPartitionsRDD[35] at csv at NativeMethodAccessorImpl.java:0), which has no missing parents
jovyan@jupyter-ec232341-~/cw$ oc logs task1-starter-kit-93be529b319792f7-driver | less
jovyan@jupyter-ec232341-~/cw$ oc logs task1-starter-kit-93be529b319792f7-driver | sed -n '/root/,/^[^ ]/p'
root
-- actor: struct (nullable = true)
|   |-- avatar_url: string (nullable = true)
|   |-- display_login: string (nullable = true)
|   |-- gravatar_id: string (nullable = true)
|   |-- id: long (nullable = true)
|   |-- login: string (nullable = true)
|   |-- url: string (nullable = true)
-- created_at: string (nullable = true)
-- id: string (nullable = true)
-- org: struct (nullable = true)
|   |-- avatar_url: string (nullable = true)
|   |-- gravatar_id: string (nullable = true)
|   |-- id: long (nullable = true)
|   |-- login: string (nullable = true)
|   |-- url: string (nullable = true)
-- payload: struct (nullable = true)
|   |-- action: string (nullable = true)
|   |-- before: string (nullable = true)
|   |-- comment: struct (nullable = true)
|   |   |-- _links: struct (nullable = true)
|   |   |   |-- html: struct (nullable = true)
|   |   |   |   |-- href: string (nullable = true)
|   |   |   |-- pull_request: struct (nullable = true)
|   |   |   |   |-- href: string (nullable = true)
|   |   |   |-- self: struct (nullable = true)
|   |   |   |   |-- href: string (nullable = true)
|   |-- size: long (nullable = true)
-- public: boolean (nullable = true)
-- repo: struct (nullable = true)
|   |-- id: long (nullable = true)
|   |-- name: string (nullable = true)
|   |-- url: string (nullable = true)
-- type: string (nullable = true)
-- created_at_ts: timestamp (nullable = true)
-- date: string (nullable = true)
-- hour: integer (nullable = true)
-- dow: integer (nullable = true)
```

Row count: 157160552

- You can programmatically configure CodeRabbit by adding a ``.coderabbit.yaml`` file to the root of your repository.
- Please see the [configuration documentation](https://docs.coderabbit.ai/guides/configure-coderabbit) for more information.
jovyan@jupyter-ec232341-~/cw\$

```
oc logs -f $(oc get pod -l spark-app-name=task1\ --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')
```

I used the above command to obtain the Spark log output. In this output, there was a large volume of lines and so to clearly obtain the summary metrics required I had to use terminal filtering with grep.

Summary metrics:

```
2025-12-18 15:25:45,119 INFO util.ShutdownHookManager: Deleting directory /tmp/spark-4250e840-fdc3-44e8-91c9-62dab33826a6
2025-12-18 15:25:45,124 INFO impl.MetricsSystemImpl: Stopping s3a-file-system metrics system...
2025-12-18 15:25:45,124 INFO impl.MetricsSystemImpl: s3a-file-system metrics system stopped.
2025-12-18 15:25:45,124 INFO impl.MetricsSystemImpl: s3a-file-system metrics system shutdown complete.
jovyan@jupyter-ec23234:~/cw$ oc logs task1-starter-kit-93be529b319792f7-driver | grep Distinct
Distinct event types: 15
Distinct repositories: 11700748
Distinct actors: 6080031
jovyan@jupyter-ec23234:~/cw$ oc logs task1-starter-kit-93be529b319792f7-driver | grep "Row count"
Row count: 157160552
jovyan@jupyter-ec23234:~/cw$
```

10 row preview:

Due to terminal display constraints, the full top-10 table for this question is too large and messy, thus I am unable to include a full screenshot of it. However I have included the 1st and 10th row from the table with the timestamps from the thermal logs as a preview below and the full table is available in the logs.

```
2025-12-23 23:58:05,077 INFO codegen.CodeGenerator: Code generated in 38.991311 ms
```

```
pos/kaiyunsports-cn/kaiyunty] |true |[877846025, kaiyunsports-cn/kaiyunty, https://api.github.com/re
IssuesEvent |2024-10-29 16:00:00|2024-10-29|16 |3 |
|[https://avatars.githubusercontent.com/u/167532951?, Ruslan-Nikiforov, , 167532951, Ruslan-Nikiforov, https://api.github.com/user
s/Ruslan-Nikiforov] |2024-10-29T16:00:00Z|43318224869|null
|[, 4102689f39083682405b4a4cc5a2573989cbc2ae,, [[167532951+ruslanavarra@u
sers.noreply.github.com, Ruslan Nikiforov], true, Update README.md, f8cdea272bed854622ad17559c7da210ab68bb56, https://api.github.c
om/repos/ruslanavarra/Yandex.Practicum.-Data-Science/commits/f8cdea272bed854622ad17559c7da210ab68bb56]], 1,, f8cdea272bed854622ad
17559c7da210ab68bb56,,,,, 20953775354,, refs/heads/main,, 814875414,, 1]
```

Question 2:

Explanation:

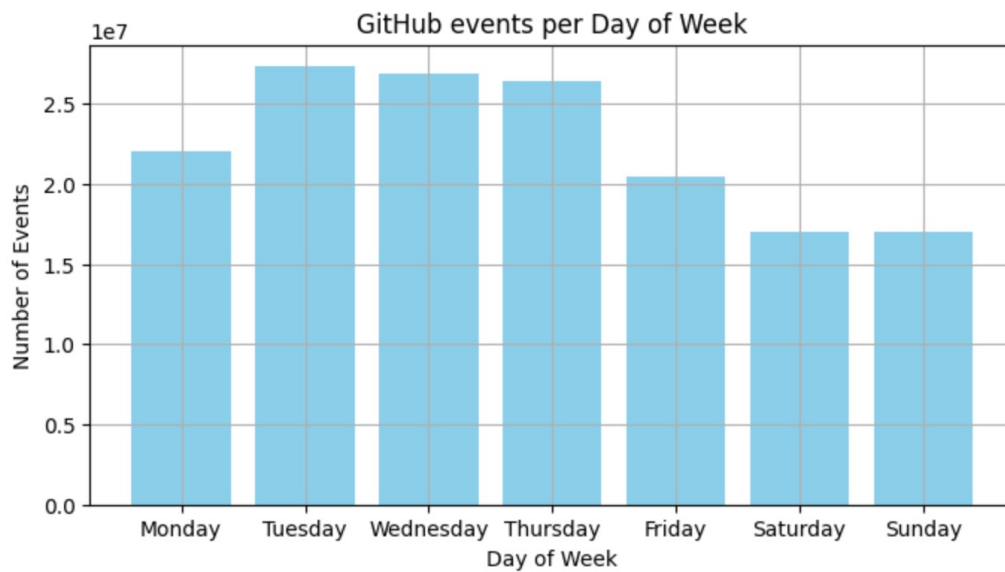
- The groupBy spark operation was used to aggregate events per day of the week and count("*") computes the total number of events each day
- The when operation was used to assign the name of the day of the week for the output results table. withColumn was used to add the column
- Spark operation orderBy was used to ensure the ordering of Monday -> Sunday in the table output. This helped to ensure accurate plots can be obtained

Table:

2025-12-24 00:26:30,107 INFO codegen.CodeGenerator: Code generated in 7.207361 ms

dow	event_count	Day of Week
2	22069316	Monday
3	27308764	Tuesday
4	26842974	Wednesday
5	26374477	Thursday
6	20483135	Friday
7	17044941	Saturday
1	17036945	Sunday

Bar Chart:



Trend analysis:

There is much higher GitHub activity during Monday – Friday, which would mirror a normal working week where people tend to use GitHub for work-related projects. GitHub activity decreases during the weekend, which reinforces the idea that many people are likely to not be working during the weekend, hence less GitHub activity.

Question 3a:

Explanation:

- Using the filter Spark operation the events can be filtered to only show WatchEvent and PullRequestEvent
- The groupBy operation was used to group the events by repository name and event type and then the count("*") was used to count the number of events for each event type
- To produce the top 10 table ordered by event count in descending order, orderBy and limit Spark operations were used

Table:

2025-12-24 00:35:51,273 INFO codegen.CodeGenerator: Code generated in 8.203391 ms

type	name	event_count
PullRequestEvent	Nix-Drone/mergequeue-github	78054
PullRequestEvent	RishabhGupta34/gitX_Harness	69637
PullRequestEvent	leandrohstein/transporteservico-urbs-data	30009
PullRequestEvent	google-test/signclav2-probe-repo	25192
PullRequestEvent	mhutchinson/mhutchinson-distributor	21407
PullRequestEvent	microsoft/winget-pkgs	21257
PullRequestEvent	actions-canary/ForkPRCanary	16525
WatchEvent	openai/swarm	14617
PullRequestEvent	MetaMask/eth-phishing-detect	12559
PullRequestEvent	NixOS/nixpkgs	11619

Question 3b:

Explanation:

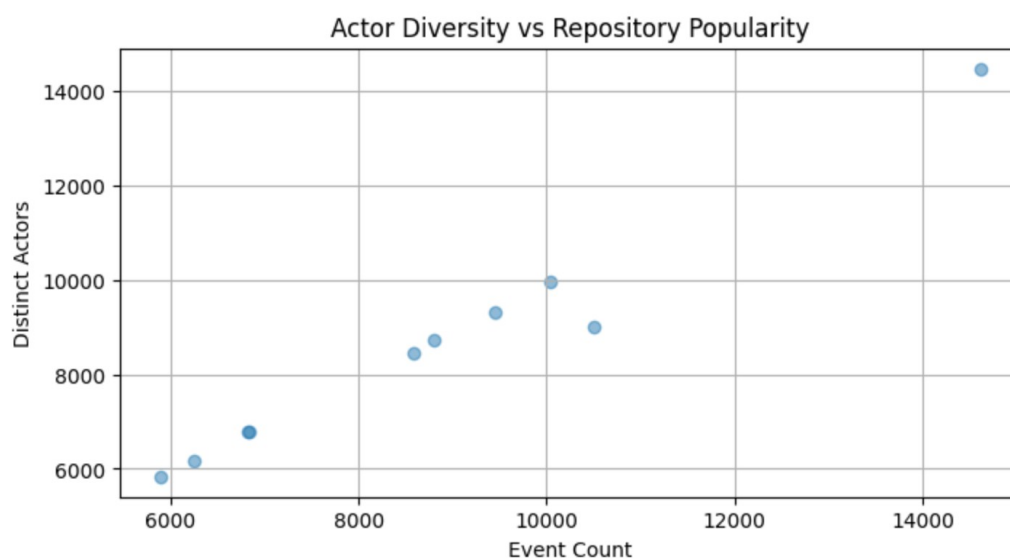
- To count distinct actors per repository countDistinct() function was used
- To produce the top 10 table ordered by distinct actors in descending order, orderBy and limit Spark operations were used

Table:

2025-12-24 00:45:41,734 INFO codegen.CodeGenerator: Code generated in 14.682841 ms

type	name	distinct_actors	event_count
WatchEvent	openai/swarm	14472	14617
WatchEvent	microsoft/BitNet	9953	10044
WatchEvent	TheAlgorithms/Python	9300	9448
WatchEvent	shardeum/shardeum	9020	10499
WatchEvent	hcengineering/platform	8717	8796
WatchEvent	feder-cr/Auto_Jobs_Applier_AIHawk	8448	8590
WatchEvent	unclecode/crawl4ai	6776	6834
WatchEvent	3b1b/manim	6772	6819
WatchEvent	HKUDS/LightRAG	6168	6244
WatchEvent	SWivid/F5-TTS	5838	5899

Scatter Plot:



Insight:

There tends to be a positive correlation, repositories with more events tend to have more actors. However, there is an outlier which tells us this may not always be the case and some repositories that have many events could still have a low number of contributors. So the plot shows the events are likely to be related but the correlation isn't very strong.

Question 4:

Explanation:

- Used Window.partitionBy Spark operation to define a window partitioned by each actor and ordered by a timestamp using orderBy
- The row_number() Spark function is used to assign each actor a row number, and the filter operation was used to filter where row_number = 1 to identify the new contributors
- The new contributors are identified and counted using countDistinct(actor.id) and then grouped by the repository name using groupBy
- The orderBy and limit operations are once again used to formulate the Top-10 table in descending order for the new contributors' value

Table:

```
2025-12-24 01:00:59,149 INFO scheduler.DAGScheduler: Job 20 finished: showString at NativeMethodAccessorImpl.java:0, took 611.326343 s
```

name	new_contributors
shardeum/shardeum	8448
Flowseal/zapret-discord-youtube	4082
TheAlgorithms/Python	3866
feder-cr/Auto_Jobs_Applier_AIHawk	3566
openai/swarm	3359
bia-pain-bache/BPB-Worker-Panel	2857
Redot-Engine/redot-engine	2696
home-assistant/core	2549
unclecode/crawl4ai	2537
sindresorhus/awesome	2219

Summary:

The repository shardeum will attract the most new contributors and attract more than double the number of new contributors than Flowseal/zapret-discord-youtube who are in 2nd. This shows that shardeum is extremely popular among new contributors. However, repositories such as sindresorhus/awesome are less likely to attract new contributors.

Question 5:

Explanation:

- To obtain the value of the total commits I first needed to filter for PushEvent as the commit count correlates to the payload.size. So withColumn Spark operation is used to extract the commit count and cast to an integer using the cast API
- The commits are summed using the spark_sum operation and they are grouped by hourly activity using groupBy

Table:

2025-12-24 01:16:44,376 INFO codegen.CodeGenerator: Code generated in 12.357067 ms
2025-12-24 01:16:44,387 INFO codegen.CodeGenerator: Code generated in 7.101069 ms

hour	total_commits
0	12381602
1	12863569
2	11283547
3	10898523
4	10219895
5	10811248
6	12920381
7	15021421
8	16184762
9	15517980
10	14328207
11	14874937
12	17057448
13	18029441
14	18372712
15	17429585
16	16078730
17	16545847
18	16489444
19	15801715
20	14658948
21	15871058
22	13154243
23	12217706

Line Chart:



Interpretation:

The number of commits is generally low during the early hours of the morning and increases throughout the day where it peaks between 12:00 – 15:00 which is during the typical working hours and so productivity is likely to be higher. Total commits tend to dip after 17:00 as most people have finished work.

Question 6:

Explanation:

- To filter events with non-null org.login I used `df.filter(col("org.login").isNotNull())`
- To group the organisations the `groupBy` Spark operation was used
- The event count was computed using the `count` operation and to obtain the top-5 in descending count order, `orderBy` and `limit` operations were used

Table:

```
2025-12-24 01:26:53,211 INFO scheduler.DAGScheduler: Job 25 finished: showString at NativeMethodAccessorImpl.java:0, took 310.873427 s
+-----+-----+
|login|event_count|
+-----+-----+
|static-web-apps-testing-org|2133796|
|magicwave-io|457629|
|microsoft|345886|
|minder-smoke-tests|305370|
|MetadataGitTesting|261238|
+-----+-----+
```

Insight:

The organisation static-web-apps-testing-org contributes the highest number of GitHub events, whereas MetaDataGitTesting contributes a much lower number of GitHub events. This suggests organisations contribute at a much larger scale due to the greater abundance of resources in comparison to individual contributors.

APIs used:

The `DataFrame(Spark SQL)` API is used as we require loading and processing data into DataFrames and these are analysed using many of the operations as stated in the explanations.

The `SparkSession` API was required to start `Spark(SparkSession.builder)` and access the `DataFrame(spark.read)`.

Challenges encountered:

The datasets that were being analysed were extremely large, and so testing Spark scripts could be time-consuming. To overcome this, the Python script was tested locally to ensure the logic was correct before testing on the full dataset. To obtain the graphs and plots, matplotlib had to be run on Jhub and not on the cluster thus, output CSV files were obtained and copied to my personal bucket and then these were processed in the notebook to obtain the plots and graphs.

Insight and knowledge gained:

I was able to produce tables that summarised large amounts of data using Spark APIs such as `groupBy` and aggregations. These tables were important for the summary, analysis and producing plots/graphs. I was able to use matplotlib to effectively construct graphs and plots to make these large datasets much easier to interpret.

Task1 Spark App ID:

Version	App ID	App Name	Started	Completed	Duration	Spark User	Last Updated
3.0.1	spark-ad831472e06a456683da942b183e6ff2	task1	2025-12-23 21:54:29	2025-12-24 01:31:44	3.6 h	ec23234	2025-12-24 01:31:45

Task 2:

Question 1:

Explanation:

- The Spark operations withColumn, to_timestamp, to_date, hour and month were used for time parsing. This works by converting a string form of a date/time into a timestamp and this allowed me to extract the month, date and hour
- The filter operation was used to remove missing station IDs and any null or negative duration values. To check for null values, the isNotNull Spark operation was used. To ensure that the duration was an integer, cast was used
- After the data was cleaned, the DataFrame.cache operation was used so the data can be used multiple times without recomputation
- To obtain the summary statistics I used agg to aggregate functions and then applied operations such as countDistinct, min and max to obtain the necessary metrics. The schema was printed using printSchema

DataFrame schema and Row count:

```
2025-12-28 22:11:23,099 INFO scheduler.DAGScheduler: Job 2 finished: count at NativeMethodAccessorImpl.java:0, took 41.265549 s
Row Count: 1326040
Schema:
root
|-- Number: string (nullable = true)
|-- Start date: string (nullable = true)
|-- Start station number: string (nullable = true)
|-- Start station: string (nullable = true)
|-- End date: string (nullable = true)
|-- End station number: string (nullable = true)
|-- End station: string (nullable = true)
|-- Bike number: string (nullable = true)
|-- Bike model: string (nullable = true)
|-- Total duration: string (nullable = true)
|-- Total duration (ms): string (nullable = true)
|-- StartTime: timestamp (nullable = true)
|-- month: integer (nullable = true)
|-- month_name: string (nullable = true)
|-- date: date (nullable = true)
|-- hour: integer (nullable = true)
|-- Duration: integer (nullable = true)
```

Summary Statistics table:

```
2025-12-28 22:11:27,212 INFO codegen.CodeGenerator: Code generated in 13.894647 ms
```

Total Rows	Distinct Start Stations	Distinct End Stations	Min Duration (in s)	Max Duration (in s)	Start Date	End Date
1326040	802	804	1	2671322	2024-08-01	2024-09-17

Data quality validation:

The data from the CSV was validated by parsing timestamps and using them to extract other metrics for time-based analysis. Records with missing station IDs were removed, and any negative or null duration values were also filtered out. The schema inspection and log output were used to confirm that the dataset is cleaned accurately and ready for analysis, indicating high data quality.

Question 2:

Explanation:

- To get the overall top-k stations I used `groupBy`, `count` and `orderBy` operations to obtain the list
- To group these results by month, I used `Window.partitionBy()` to group trips by month and station. The window function also allowed ranking stations within each month
- To construct the routes in part b, the `concat` and `lit` functions were used to combine the start station and end station names into a single string. Self-loop routes were removed using a filter operation

Top-K stations:

2025-12-28 22:11:34,853 INFO codegen.CodeGenerator: Code generated in 9.631886 ms

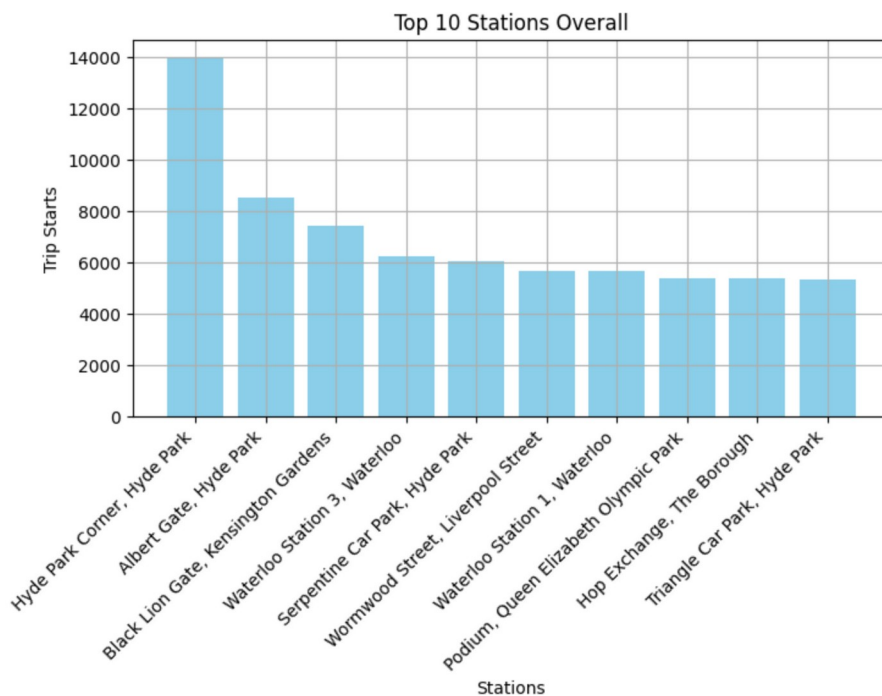
Start station number	Start station	Trip Starts
1075	Hyde Park Corner, Hyde Park	13953
1132	Albert Gate, Hyde Park	8519
1157	Black Lion Gate, Kensington Gardens	7428
1072	Waterloo Station 3, Waterloo	6214
1217	Serpentine Car Park, Hyde Park	6034
2587	Wormwood Street, Liverpool Street	5663
2696	Waterloo Station 1, Waterloo	5658
300021	Podium, Queen Elizabeth Olympic Park	5383
960	Hop Exchange, The Borough	5367
1088	Triangle Car Park, Hyde Park	5343

Monthly Top-K table:

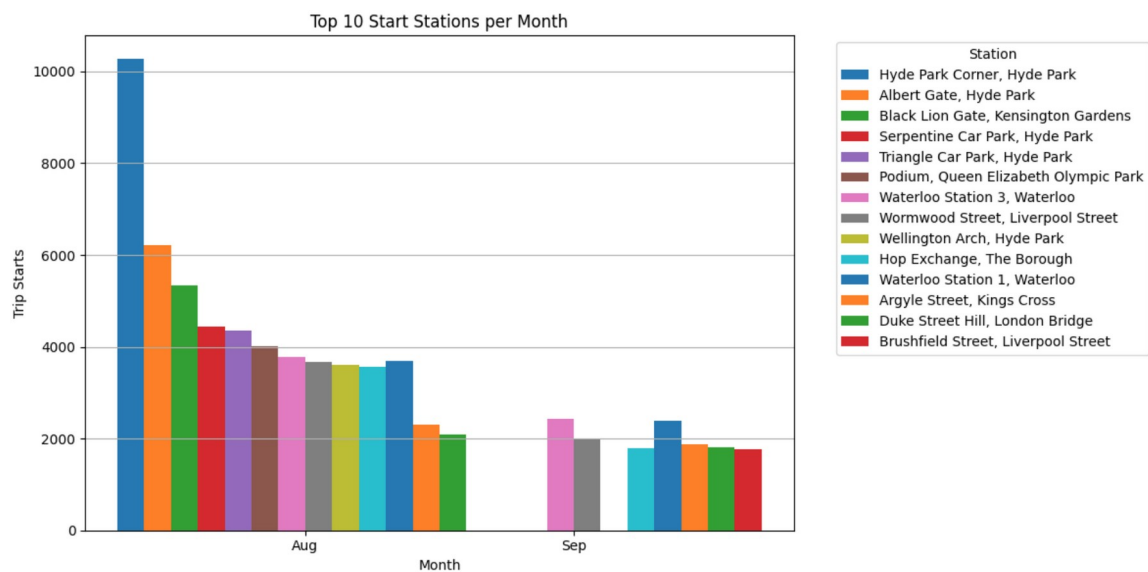
2025-12-28 22:11:39,223 INFO codegen.CodeGenerator: Code generated in 7.403996 ms

month	month_name	Start station number	Start station	Trip Starts	Rank
8	Aug	1075	Hyde Park Corner, Hyde Park	10268	1
8	Aug	1132	Albert Gate, Hyde Park	6217	2
8	Aug	1157	Black Lion Gate, Kensington Gardens	5331	3
8	Aug	1217	Serpentine Car Park, Hyde Park	4449	4
8	Aug	1088	Triangle Car Park, Hyde Park	4344	5
8	Aug	300021	Podium, Queen Elizabeth Olympic Park	4020	6
8	Aug	1072	Waterloo Station 3, Waterloo	3787	7
8	Aug	2587	Wormwood Street, Liverpool Street	3661	8
8	Aug	1109	Wellington Arch, Hyde Park	3600	9
8	Aug	960	Hop Exchange, The Borough	3571	10
9	Sep	1075	Hyde Park Corner, Hyde Park	3685	1
9	Sep	1072	Waterloo Station 3, Waterloo	2427	2
9	Sep	2696	Waterloo Station 1, Waterloo	2380	3
9	Sep	1132	Albert Gate, Hyde Park	2302	4
9	Sep	1157	Black Lion Gate, Kensington Gardens	2097	5
9	Sep	2587	Wormwood Street, Liverpool Street	2002	6
9	Sep	1011	Argyle Street, Kings Cross	1877	7
9	Sep	300083	Duke Street Hill, London Bridge	1819	8
9	Sep	960	Hop Exchange, The Borough	1796	9
9	Sep	1161	Brushfield Street, Liverpool Street	1766	10

Top-K stations Bar chart:



Top-K stations per month:



Commuter vs tourist station popularity analysis:

Stations near tourist locations such as Hyde Park and Olympic Park will have a large number of visitors during August, as shown by the data. However, in September, this isn't a peak time for tourists, and so the most popular stations consist of stations located near major business districts such as Liverpool Street, Waterloo and King's Cross, reflecting a more commuter-driven pattern.

Top-10 Routes:

2025-12-28 22:11:52,468 INFO codegen.CodeGenerator: Code generated in 12.202434 ms

Route	Trips
Hyde Park Corner, Hyde Park -> Albert Gate, Hyde Park	898
Hyde Park Corner, Hyde Park -> Serpentine Car Park, Hyde Park	727
Albert Gate, Hyde Park -> Hyde Park Corner, Hyde Park	673
Hyde Park Corner, Hyde Park -> Triangle Car Park, Hyde Park	648
Ackroyd Drive, Bow -> Maplin Street, Mile End	615
Hyde Park Corner, Hyde Park -> Black Lion Gate, Kensington Gardens	570
Maplin Street, Mile End -> Ackroyd Drive, Bow	542
Triangle Car Park, Hyde Park -> Hyde Park Corner, Hyde Park	510
Black Lion Gate, Kensington Gardens -> Hyde Park Corner, Hyde Park	505
Hyde Park Corner, Hyde Park -> Queen's Gate, Kensington Gardens	482

Mobility patterns analysis:

The most frequent routes heavily involve stations near Hyde Park and Kensington Gardens, reflecting high tourism during the summer months, where the hotter weather will lead to increased cycling in scenic areas. Routes in more commuter-driven areas such as Stratford and Hackney appear as very frequent routes due to weekday commuting patterns which tend to spike in September as people return to work after their holidays

Question 3:

Explanation:

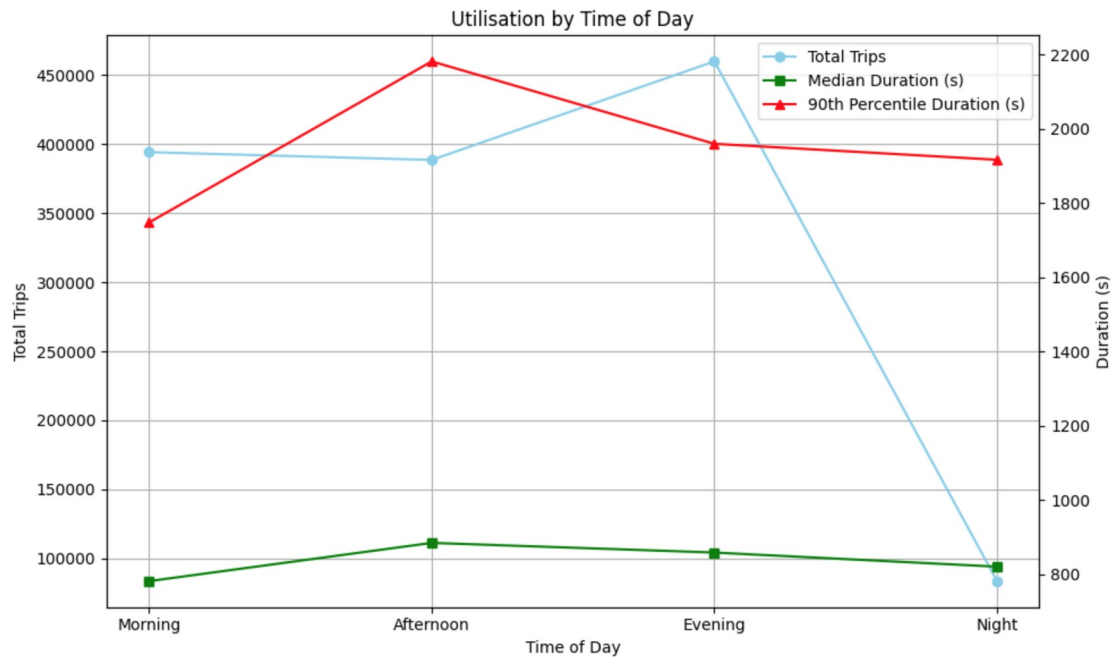
- To create the hour-based time-of-day buckets, I used when and withColumn operations to map the hours into 4 buckets.
- To aggregate the statistics for each bucket, the percentile_approx function via expr() was required to calculate the median and any percentiles.

Table of metrics:

2025-12-28 22:12:00,216 INFO codegen.CodeGenerator: Code generated in 7.10946 ms

time_of_day	Trips	Median (s)	90th Percentile (s)
Morning	394106	782	1748
Afternoon	388462	885	2182
Evening	459782	859	1960
Night	83690	821	1917

Line chart:



Commuting and leisure pattern differences:

Morning, afternoon and evening show high volumes of trips, which aligns with commuter usage during peak travel times. At night, the number of trips is much lower as this is off-peak time for work and so any form of cycling is likely to be more leisure-oriented. The afternoon has the highest 90th percentile, which indicates more people may favour this time of day for more recreational cycling in comparison to other times.

Question 4:

Explanation:

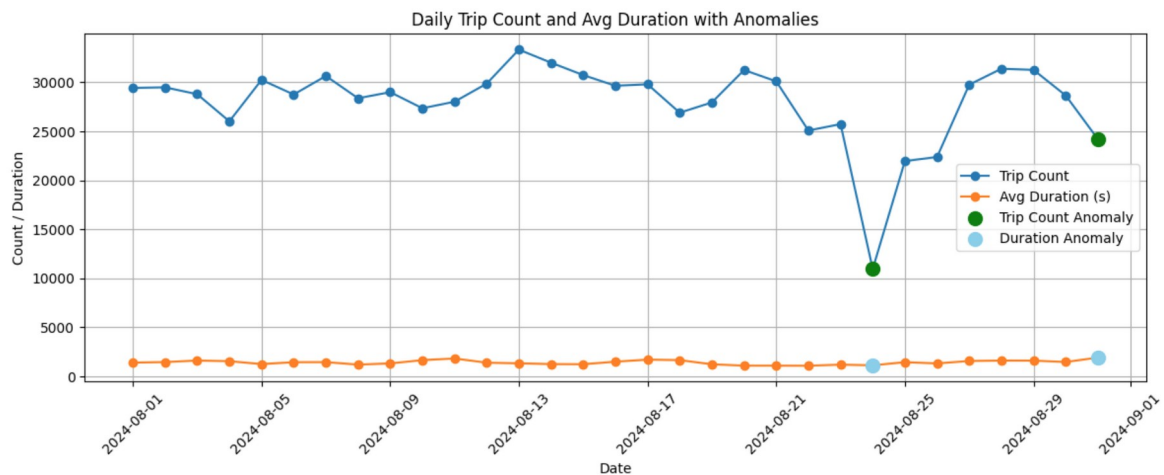
- The cleaned data was filtered to show only the data in August and the count and agg operations were used to compute the total number of trips in just august
- To compute the mean and standard deviation, the mean and stddev functions were required, these values can then be used for z-score calculation
- The correct z-score for each metric was calculated using the withColumn operation and a Boolean field(is_anomaly) was used to flag any anomalies in the required table which was ordered by date using the orderBy Spark operation

Table of flagged anomalies:

2025-12-28 22:12:07,913 INFO codegen.CodeGenerator: Code generated in 8.029313 ms

date	Avg_Duration_(s)	Trip_Count	zscore_duration	zscore_trip_count	is_anomaly
2024-08-01	1409.7273870200477	29430	-0.07622023886231878	0.33651489075751195	false
2024-08-02	1465.6408056695263	29491	0.18016717057303103	0.3512704827072048	false
2024-08-03	1619.5349749930535	28792	0.8858390531566602	0.182185912660724	false
2024-08-04	1555.0133722717492	26024	0.5899793655557791	-0.4873793089253402	false
2024-08-05	1259.4185785123966	30250	-0.7654516784596906	0.5348687497533835	false
2024-08-06	1452.2738968670676	28759	0.11887406414529655	0.17420337931089014	false
2024-08-07	1458.8024860526573	30651	0.14881049280572842	0.6318686247013646	false
2024-08-08	1206.4820088105728	28375	-1.0081889372084183	0.08131571851282343	false
2024-08-09	1328.214509844488	29001	-0.44999233006065315	0.23274195720967178	false
2024-08-10	1662.479711946191	27356	1.0827594011203971	-0.1651752355320463	false
2024-08-11	1830.6262132457894	28024	1.8537844415330114	-0.0035894089354094193	false
2024-08-12	1399.9198219008404	29871	-0.12119220220345683	0.44319056370529164	false
2024-08-13	1336.7858064129098	33339	-0.4106891971855853	1.2820822502878317	false
2024-08-14	1260.221989626945	32006	-0.7617676881162998	0.9596362819445428	false
2024-08-15	1244.084574848741	30742	-0.8357647727142744	0.6538810651509065	false
2024-08-16	1504.20239460371	29650	0.3569888732939105	0.38973177975640433	false
2024-08-17	1712.7664407462087	29804	1.3133459836872772	0.42698360205562896	false
2024-08-18	1663.0676132773297	26903	1.0854551851188534	-0.27475364787976564	false
2024-08-19	1238.655754713606	27951	-0.8606582806477359	-0.021247740285041894	false
2024-08-20	1100.4798771514493	31258	-1.4942549404532575	0.7786988593483086	false
2024-08-21	1099.852672491114	30103	-1.4971309473570251	0.49931019210412364	false
2024-08-22	1092.0096862917048	25087	-1.5330944595598772	-0.7140348770706227	false
2024-08-23	1200.6923853923854	25740	-1.0347368855479917	-0.5560774747239103	false
2024-08-24	1133.2014230979748	10962	-1.3442123817869278	-4.130801045749509	true
2024-08-25	1453.688014930129	21969	0.12535841218877658	-1.468263331154925	false
2024-08-26	1327.7954606380126	22382	-0.451913853427838	-1.3683607168070042	false
2024-08-27	1575.6944425777838	29762	0.6848110923252526	0.4168240141558404	false
2024-08-28	1617.7348115581892	31389	0.8775845187215814	0.8103870977976492	false
2024-08-29	1611.8328163474146	31271	0.8505212967054747	0.7818434936982432	false
2024-08-30	1471.416492183138	28656	0.20665121212696982	0.1492881994614087	false
2024-08-31	1924.5221432702635	24206	2.2843382305333964	-0.9271443280161873	true

Line chart:



Anomaly causation analysis:

The anomalies observed could be a result of many factors, as they lie in the late summer period, where hotter temperatures and more public events will lead to more frequent leisure-oriented cycling and the longer days can result in longer duration of cycling trips.

Question 5:

Explanation:

- The withColumn and when operations were used to define the morning and evening peak windows. Then for each station in each peak window, the number of trips starting and ending at the station is computed. The trips are grouped by peak_window and EndStation/StartStation Name
- To compute the net flow, starts and ends are joined using the join operation, and missing values are replaced with 0 using fillna(0)
- A window function is then used to rank the stations in each peak window by net flow using orderBy and row_number().over
- The top-10 stations with positive and negative net flow are obtained using the filter operation

Net inflow table:

2025-12-28 22:12:16,577 INFO codegen.CodeGenerator: Code generated in 11.828168 ms

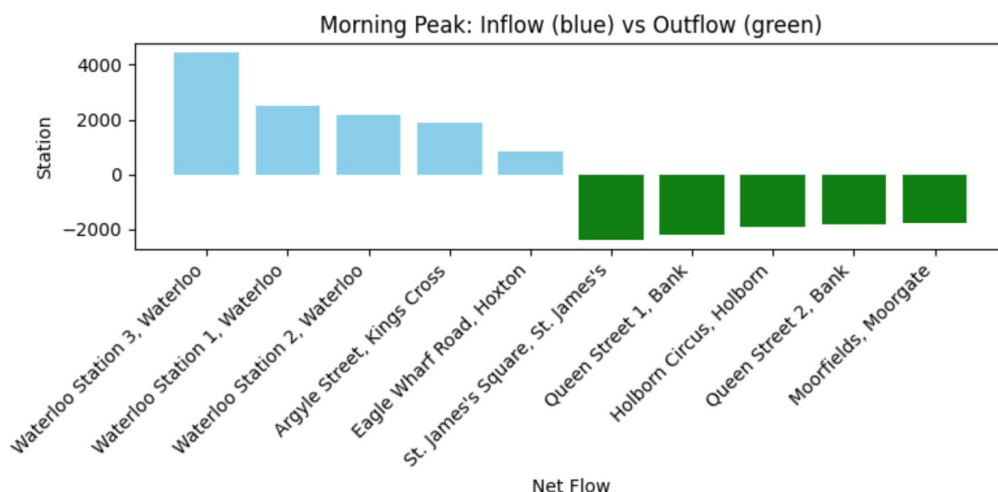
peak_window	Station Name	Starts	Ends	Net Flow	Rank
Evening	Cheapside, Bank	2167	711	1456	4
Evening	Moorfields, Moorgate	1937	673	1264	5
Evening	St. James's Square, St. James's	2062	806	1256	6
Evening	Newgate Street , St. Paul's	1542	322	1220	7
Evening	Queen Street 1, Bank	1974	774	1200	8
Evening	Queen Street 2, Bank	2072	896	1176	9
Evening	Holborn Circus, Holborn	1443	316	1127	10
Morning	Waterloo Station 3, Waterloo	4593	169	4424	1
Morning	Waterloo Station 1, Waterloo	3283	771	2512	2
Morning	Waterloo Station 2, Waterloo	2348	198	2150	5
Morning	Argyle Street, Kings Cross	2583	679	1904	7

Net outflow table:

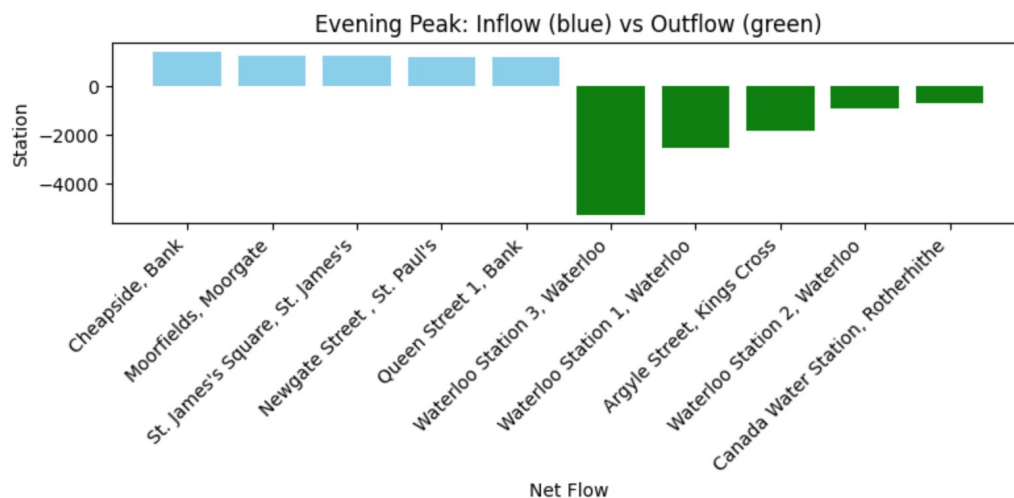
2025-12-28 22:12:20,284 INFO scheduler.DAGScheduler: Job 28 finished: showString at Native

peak_window	Station Name	Starts	Ends	Net Flow	Rank
Evening	Waterloo Station 3, Waterloo	419	5699	-5280	1
Evening	Waterloo Station 1, Waterloo	889	3418	-2529	2
Evening	Argyle Street, Kings Cross	1066	2889	-1823	3
Morning	St. James's Square, St. James's	267	2634	-2367	3
Morning	Queen Street 1, Bank	543	2729	-2186	4
Morning	Holborn Circus, Holborn	229	2159	-1930	6
Morning	Queen Street 2, Bank	343	2150	-1807	8
Morning	Moorfields, Moorgate	519	2304	-1785	9
Morning	Soho Square , Soho	268	1825	-1557	10

Morning Inflow v Outflow Bar chart:



Evening Inflow v Outflow Bar chart:



Major commuter transport hubs such as King's Cross and Waterloo experience greater commuter transport pressure as they have a large net outflow in the morning and a large net inflow in the evening. This represents their daily commuting behaviour centred around their working hours.

Spark APIs used:

The DataFrame(Spark SQL) API is used to load and process data from CSV files into DataFrames and these are analysed using many of the operations such as withColumn, filter, etc. Built-in functions date_format, mean and stddev are important for parsing and computations.

The SparkSession API was required to start Spark(SparkSession.builder) and access the DataFrame(spark.read).

Challenges encountered:

One initial challenge faced during this task was to ensure correct data cleaning to prevent datasets from being incorrectly processed, leading to inconsistent and unreliable results. Incorrect timestamp parsing or incorrect naming of dataset columns can very easily result in empty DataFrames or data being incorrectly filtered out. I faced this in question 1, where I incorrectly parsed timestamps, and I had to carry out debugging using df.show function and schema inspection to rectify the error.

Another challenge faced was the calculation of percentiles using the percentile_approx function. My code wasn't running as percentile_approx wasn't supported in my PySpark server, thus after research I was able to fix this and get the desired results by using the percentil_approx via expr.

Insight and knowledge gained:

I understand the importance of correctly parsing timestamps and using them to derive other metrics for time-based analysis accurately. This task also reinforced my abilities to accurately clean datasets, such as handling null values and filtering invalid records, which is vital to ensure results are accurate and reliable. Using window functions and grouped

aggregations allowed me to partition stations monthly and rank them by the number of trips.

Task2 Spark App ID:

Version	App ID	App Name	Started	Completed	Duration	Spark User	Last Updated
3.0.1	spark-7a7d7dd6d0ac49d6b17689da74571550	task2	2025-12-28 22:10:12	2025-12-28 22:12:24	2.2 min	ec23234	2025-12-28 22:12:25

Task 3:

Question 1:

Explanation:

- Spark.read.csv is used to load the CSV files for airports and routes into DataFrames so they can be processed.
- To calculate the vertices and edges the count operation is needed. The select, withColumn and cast are used to select the necessary columns of the datasets that will be used for processing
- To clean the dataset, by removing null fields and routes that have no endpoints, the dropna and dropDuplicates functions were used. This is to prevent inconsistencies and invalid results
- GraphFrames were created using: g = GraphFrame(df_vertices, df_clean_edges)
- To ensure a clean and readable way for visualising the summary metrics Spark.createDataFrame was used and the show operation was used to display results in the logs

Vertices Preview Table:

2026-01-02 18:18:39,951 INFO codegen.CodeGenerator: Code generated in 20.959628 ms

id	name	city	country	iata	lat	lon
148	Hall Beach Airport	Hall Beach	Canada	YUX	68.77610015869999	-81.2425
463	Varkaus Airport	Varkaus	Finland	VRK	62.171100616455	27.868600845337
471	Leicester Airport	Leicester	United Kingdom	\N	52.6077995300293	-1.03193998336792
496	Lasham Airport	Lasham	United Kingdom	QLA	51.187198638916016	-1.0334999561309814
833	Mkuze Airport	Mkuze	South Africa	MZQ	-27.626100540161133	32.0443000793457
1088	Tambacounda Airport	Tambacounda	Senegal	TUD	13.736800193786621	-13.65310001373291
1238	Salamanca Airport	Salamanca	Spain	SLM	40.95209884643555	-5.501989841461182
1342	Valence-Chabeuil Airport	Valence	France	VAF	44.9216	4.9699
1580	Plzeň-Líně Airport	Line	Czech Republic	\N	49.675201416016	13.274600028992
1591	Beersheba (Teyman) Airport	Beer-sheba	Israel	BEV	31.287000656128	34.722999572754

only showing top 10 rows

Edges Preview Table:

2026-01-02 18:18:43,579 INFO codegen.CodeGenerator: Code generated in 17.905078 ms

src	dst	airline	stops
2965	2990	2B	0
2966	2990	2B	0
2966	2962	2B	0
2968	2990	2B	0
2968	4078	2B	0
4029	2990	2B	0
4029	6969	2B	0
4029	6160	2B	0
6156	2952	2B	0
6156	2990	2B	0

only showing top 10 rows

```
2026-01-02 18:18:44,991 INFO codegen.CodeGenerator: Code generated in 16.081063 ms
```

Metric	Value
Raw Vertices	7698
Clean Vertices	7698
Raw Edges	67663
Edges Removed (no endpoint)	892
Clean Edges	66771

Data Quality Issues:

From the summary table we can see that Raw Vertices == Clean Vertices which tells us the quality of the airport dataset was good and it was well structured with no null fields. However, we can see that 892 edges were removed from the routes dataset after cleaning which means the dataset contained routes with missing endpoints. It is important that these routes are removed before processing as these would introduce inconsistencies, as some routes wouldn't have the corresponding endpoint vertices. Thus, the cleaning was important to maintain high data quality and obtain reliable results

Question 2a:

Explanation:

- I used `g.connectedComponents` to select the groups of airports that can be reached from each other. Aggregation was carried out using `groupBy` and `count` operations to compute the size of each component
- The table of the top 5 was produced using the `select` and `order by` operations and the `limit(5)` function ensures only 5 are displayed

Number of Connected Components:

```
2026-01-02 18:27:00,210 INFO scheduler.DAGScheduler: Job 49 finished: count at NativeMethodAccessorImpl.java:0, took 6.236420 s
Number of connected components: 4491
2026-01-02 18:27:00,384 INFO datasources.FileSourceStrategy: Pruning directories with:
```

Top 5 Component Sizes:

```
2026-01-02 18:27:06,929 INFO codegen.CodeGenerator: Code generated in 9.359038 ms
```

Component ID	size
1	3188
1998	10
3860	4
3726	4
5642	4

Global Graph Connectivity:

The table for Top 5 Component Sizes tells us there is one major component which is very highly interconnected and will contain a vast majority of airports. The remaining components are only connected to a handful of airports; these are likely to be represented by airports in very remote or isolated areas.

Question 2b:

Explanation:

- Degree computation was carried out using `g.inDegrees` and `g.outDegrees` and the total degree was computed using the `withColumn` and `join` spark operations. This helps us distinguish incoming and outgoing routes
- The airports with the top 10 total degree were displayed in a table by using `orderBy` and `select` operation to select the correct columns. For readability purposes, the `join` operation was required to include the name of the airport
- Triangle count was computed using `g.triangleCount` which helps us count the number of triangles passing through each airport. The table for the top 10 airports by triangle count was computed the same way as the table for total degree.

Top 10 Airports by Degree:

```
2026-01-02 18:27:20,640 INFO scheduler.DAGScheduler: Job 56 finished: showString at NativeMethodAccessorImpl.java:0, took 3.534504 s
```

Airport	Total Degree
Hartsfield Jackson Atlanta International Airport	1826
Chicago O'Hare International Airport	1108
Beijing Capital International Airport	1061
London Heathrow Airport	1047
Charles de Gaulle International Airport	1041
Frankfurt am Main Airport	990
Los Angeles International Airport	986
Dallas Fort Worth International Airport	936
John F Kennedy International Airport	911
Amsterdam Airport Schiphol	903

Top 10 Airports by Triangle Count:

```
2026-01-02 18:28:00,099 INFO scheduler.DAGScheduler: Job 62 finished: showString at NativeMethodAccessorImpl.java:0, took 32.007578 s
```

Airport	Triangle Count
Amsterdam Airport Schiphol	4543
Frankfurt am Main Airport	4357
Charles de Gaulle International Airport	4136
Munich Airport	3658
London Heathrow Airport	3154
Atatürk International Airport	3143
Leonardo da Vinci-Fiumicino Airport	3142
Barcelona International Airport	3094
Adolfo Suárez Madrid-Barajas Airport	2755
Hartsfield Jackson Atlanta International Airport	2665

Short Commentary:

High-degree airports do tend to form many triangles as some airports within the top 10 highest degree do feature in the top 10 for highest triangle count, such as Hartsfield Atlanta, Frankfurt am and Amsterdam Airport. This suggests airports with high degrees tend to be major international hubs with large interconnections and this leads to them participating in many triangles. Regional airports such as, Barcelona International and Adolfo Suarez airports have high triangle counts indicating these airports cluster tightly due to short domestic and local routes.

Question 3:

Explanation:

- A start and goal airport is selected and their IDs are obtained using the `filter` operation to match the specific IATA code
- Breadth-First search is run using `g.bfs` up to depth 4 to find the shortest path between two selected airports

- To determine an airport hop sequence, a hop array was required and each airport is represented by a vertex column(from, v1, v2, v3, to). If statements are required to determine whether intermediate paths(v1 to v3) exist
- Use of reduce(DataFrame.unionAll, hops).dropna().distinct() is to take the BFS result and display the airport hop sequence into a table. Readability of the table is improved using the join function which includes the airport name in the table
- To compute the shortest paths reachability g.shortestPaths is used

BFS result path:

```
2026-01-02 18:29:32,698 INFO codegen.CodeGenerator: Code generated in 5.489897 ms
```

Hop	Airport	city	country
1	London Heathrow Airport	London	United Kingdom
2	Suvarnabhumi Airport	Bangkok	Thailand
3	Sydney Kingsford Smith International Airport	Sydney	Australia

This BFS result path shows that long-haul routes are most likely to be multi-hop, rather than direct flights. Large hubs in the Middle East and Asia, such as Bangkok and Dubai are likely to be stopovers, reinforcing their high global connectivity.

Question 4a:

Explanation:

- To apply the PageRank algorithm g.pageRank is used, which ranks vertices(airports) based on how influential the airport is.
- The airports are then ranked using the orderBy operation and desc() spark function

Top 10 PageRank table:

```
2026-01-02 18:30:49,788 INFO codegen.CodeGenerator: Code generated in 8.215804 ms
```

Airport	PageRank Score
Hartsfield Jackson Atlanta International Airport	62.13804254123376
Chicago O'Hare International Airport	39.10837089316784
Los Angeles International Airport	37.37815726881754
Dallas Fort Worth International Airport	35.8631611426755
Charles de Gaulle International Airport	32.791888057658596
London Heathrow Airport	32.66447425696146
Singapore Changi Airport	31.731804042892765
Beijing Capital International Airport	31.55238438567578
Denver International Airport	31.303782248659743
Frankfurt am Main Airport	29.975732873873508

Global Hub Dominance:

Airports such as Atlanta, Heathrow and Beijing have high PageRank scores, highlighting them as major global hubs and are very highly connected airports. The high number of connections to other well-connected airports and the high degree of network are factors that make these airports very influential in this field

Question 4b:

Explanation:

- Airline diversity per airport was computed using aggregation(countDistinct) which is used as the weight factor in weighted page rank.
- Weighted edges were then built using join to include airline diversity into the graph frame. PageRank was applied on the weighted graph frame.
- To compare the weighted pageRank with classic pageRank, the columns for each table were renamed using .withColumnRenamed to allow for a joint table to be built
- The function monotonically_increasing_id() was required for ranking to be established correctly in the joint table. The joint table was constructed using select and join

Classic vs Weighted PageRank for Top Airports:

2026-01-02 18:40:55,018 INFO codegen.CodeGenerator: Code generated in 9.399919 ms

Rank	Classic Top Airport	PR	Weighted Top Airport	W-PR
1	Hartsfield Jackson Atlanta International Airport	62.13804254123376	Hartsfield Jackson Atlanta International Airport	62.13804254123368
2	Chicago O'Hare International Airport	39.10837089316784	Chicago O'Hare International Airport	39.108370893167844
3	Los Angeles International Airport	37.37815726881754	Los Angeles International Airport	37.37815726881741
4	Dallas Fort Worth International Airport	35.8631611426755	Dallas Fort Worth International Airport	35.86316114267551
5	Charles de Gaulle International Airport	32.791888057658596	Charles de Gaulle International Airport	32.79188805765864
6	London Heathrow Airport	32.66447425696146	London Heathrow Airport	32.66447425696144
7	Singapore Changi Airport	31.731804042892765	Singapore Changi Airport	31.73180404289276
8	Beijing Capital International Airport	31.55238438567578	Beijing Capital International Airport	31.552384385675804
9	Denver International Airport	31.303782248659743	Denver International Airport	31.30378224865976
10	Frankfurt am Main Airport	29.975732873873508	Frankfurt am Main Airport	29.975732873873486

Summary:

These result tells us that airline diversity does increase PageRank for airports, but only slightly. This suggests an airline diversity is correlated with overall connectivity. This is because major global hubs with high connectivity are more likely to operate under many airlines.

Question 5:

Explanation:

- Label propagation was run(g.labelPropagation) to apply the community detection algorithm. The number of detected communities was detected using distinct() and count() functions
- The most popular clusters were determined using the count function and the table was produced using groupBy, withColumnRenamed and orderBy operations
- To map the communities to real-world regions the groupBy operation was used and the largest community was identified using communities.first()["Community ID"]
- Degree analysis was needed to determine the most influential hubs inside a community, and the join operation was used for table readability purposes

Number of Detected Communities:

2026-01-02 18:41:51,972 INFO scheduler.DAGScheduler: Job 218 finished: count at NativeMethodAccessorImpl.java:0, took 3.949987 s
Number of Detected Communities: 4675

Top 5 Largest Communities by Size:

2026-01-02 18:41:57,008 INFO codegen.CodeGenerator: Code generated in 10.577853 ms

Community ID	Size
3682	732
3998	662
3386	315
507	223
3316	173

#choose a large community

large_community = top5_communities.first()["Community ID"]

The above code was used to select the largest community and to extract the top hubs inside this community(Community 3682 from the table above)

Top Hubs inside largest Community:

2026-01-02 18:42:17,251 INFO scheduler.DAGScheduler: Job 229 finished: showString at NativeMethodAccessorImpl.java:0, took 4.162435 s

airport_name	degree
Hartsfield Jackson Atlanta International Airport	1826
Chicago O'Hare International Airport	1108
Los Angeles International Airport	986
Dallas Fort Worth International Airport	936
John F Kennedy International Airport	911
Miami International Airport	734
Denver International Airport	733
Lester B. Pearson International Airport	636
Charlotte Douglas International Airport	600
Philadelphia International Airport	576

Mapping of Community ID → Geographic Clusters:

2026-01-02 18:42:04,357 INFO codegen.CodeGenerator: Code generated in 10.390231 ms

label	airport_country	count
5	Papua New Guinea	23
11	Iceland	3
13	Iceland	1
14	Iceland	1
17	Iceland	1
18	Iceland	1
19	Iceland	1
20	Iceland	1
22	Canada	1
23	Canada	1
25	Canada	1
26	Canada	1
32	Canada	1
35	Canada	1
38	Canada	1
39	Canada	1
42	Canada	1
43	Canada	1
44	Canada	1
46	Canada	1

Explanatory Sentences:

Regional networks cluster this way because airports in similar geographic areas are likely to have a high number of domestic and short-haul flights and so are more highly connected. Papa New Guinea has a high number of airports and so dense clustering is likely to form here. However, countries such as Iceland and Canada appear across many small communities with a low number of airports suggesting they are isolated and less connected.

Spark APIs used:

The SparkSession API was required to start Spark(SparkSession.builder) and access the DataFrame(spark.read).

The DataFrame API is used to load csv files into DataFrames and to clean, load and process the airport and routes data these are analysed using many of the operations such as withColumn, select, dropna, and dropDuplicates. Built-in functions such as, countDistinct, count and monotonically_increasing_id are important for computations and rankings to provide clear table results.

The GraphFrames API is used for graph construction and processing, where the airports are the vertices and the flight routes are the edges. This API provides operations such as inDegrees, outDegrees, pageRank and labelPropagation. These are vital for graph-based analysis as they can be used to determine how influential airports are, research clusters in major regions and analyse different flight paths.

Challenges encountered:

During testing, I would repeatedly get an AnalysisException which references a variable being ambiguous, which would appear when joining GraphFrame outputs with airport metadata. Therefore, I had to pay extra attention to ensure metadata columns were named explicitly. Incorrect naming of dataset columns can very easily result in empty DataFrames or data being incorrectly filtered out. I faced this in question 1, where I referenced a column in the dataset but after careful schema inspection there appeared to be extra spaces before the column name which caused incorrect cleaning of data. To rectify this, I used a function to remove unnecessary spacing on column names.

Another challenge faced was referencing the edgeWeightCol when running the weighted PageRank where I received a type error due to the edgeWeightCol parameter not being available in my GraphFrames API. Thus, airline diversity was added in the graph structure and then applying PageRank, so without the edgeWeightCol parameter.

Insight and knowledge gained:

This task equipped me with the necessary skills to analyse datasets using the GraphFrames API. I was able to construct, transform and manipulate the graphs using operations such as BFS, PageRank and labelPropagation, which can then be use for evaluations of flight connectivity, connected hubs, etc.

Task3 Spark App ID:

3.0.1	spark-cc9d13f74edd4c06a015b87a4b0f4db	task3	2026-01-02 18:18:07	2026-01-02 18:42:31	24 min	ec23234	2026-01-02 18:42:34
-------	---------------------------------------	-------	---------------------	---------------------	--------	---------	---------------------

Task 4:

Question 1a:

Explanation:

The data string was identified and converted into a Spark timestamp type using regexp_extract(). This was also used to extract the other fields into 5 structured columns. The regular expressions were required to identify fields such as component and log level in the raw HDFS log file. Spark.readStream() was used to set up a streaming pipeline and spark.writeStream() was used to write the logs to the console. To inspect the schema, the printSchema operation was used.

Streaming Ingestion code snippet:

```
*****
#Streaming Ingestion and Initial Parsing
*****

#regex patterns to extract fields
#timestamp_pattern = r"^(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})"
timestamp_pattern = r"(\d{6}\s\d{6})"
log_level_pattern = r"\s(INFO|WARN|ERROR)\s"
component_pattern = r"\bdfs\.(\w+)"
host_pattern = r"(\d{1,3}(\?:\.\d{1,3}){3})(?:\:\d+)?"
message_pattern = r":\s(.*)$"

#derive fields using regexp
parsed_df = (
    logsDF
    .withColumn("timestamp", to_timestamp(regexp_extract(col("value"), timestamp_pattern, 1), "yyMMdd HHmmss"))
    .withColumn("level", regexp_extract(col("value"), log_level_pattern, 1))
    .withColumn("component", regexp_extract(col("value"), component_pattern, 1))
    .withColumn("host", regexp_extract(col("value"), host_pattern, 1))
    .withColumn("message", regexp_extract(col("value"), message_pattern, 1))
)
```

Schema screenshot:

```
2026-01-05 17:21:46,033 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@6e4dd7ae{/static/sql,null,AVAILABLE,@Spark}
root
|-- value: string (nullable = true)
|-- timestamp: timestamp (nullable = true)
|-- level: string (nullable = true)
|-- component: string (nullable = true)
|-- host: string (nullable = true)
|-- message: string (nullable = true)
```

Screenshot showing successful parsing over multiple batches:

```
Batch: 0
-----
+-----+-----+-----+-----+-----+
|value|timestamp|level|component|host|message|
+-----+-----+-----+-----+-----+
-----

Batch: 1
-----
+-----+-----+-----+-----+-----+
|value|timestamp|level|component|host|message|
+-----+-----+-----+-----+-----+
-----
|Server is working:081109 203615 148 INFO dfs.DataNode$PacketResponder: PacketResponder 1 for block blk_38865049064139660 terminating
|2008-11-09 20:36:15|INFO |DataNode | | |PacketResponder 1 for block blk_38865049064139660 terminating
|081109 204005 35 INFO dfs.FSNamesystem: BLOCK* NameSystem.addStoredBlock: blockMap updated: 10.251.73.220:50010 is added to blk_71283702376877
28475 size 67108864|2008-11-09 20:40:05|INFO |FSNamesystem|10.251.73.220|BLOCK* NameSystem.addStoredBlock: blockMap updated: 10.251.73.220:5001
0 is added to blk_7128370237687728475 size 67108864|
|081109 203807 222 INFO dfs.DataNode$PacketResponder: PacketResponder 0 for block blk_-6952295868487656571 terminating
|2008-11-09 20:38:07|INFO |DataNode | | |PacketResponder 0 for block blk_-6952295868487656571 terminating
-----

Batch: 2
-----
+-----+-----+-----+-----+-----+
|value|timestamp|level|component|host|message|
+-----+-----+-----+-----+-----+
-----
|081109 204015 308 INFO dfs.DataNode$PacketResponder: PacketResponder 2 for block blk_8229193803249955061 terminating|2008-11-09 20:40:15|INFO
|DataNode | | |PacketResponder 2 for block blk_8229193803249955061 terminating|
-----
```

Summary verifying correct parsing:

The socket source on the cluster was used to construct a Structured Streaming DataFrame. Using `regexp_extract`, each log line was parsed to derive the component, host, level and timestamp fields. The `to_timestamp()` function was used to convert a timestamp string to a Spark timestamp type. Correct parsing was able to be verified by writing the DataFrame to a console sink in append mode with truncation disabled. Thus, I was able to identify that all fields were extracted and consistent across micro-batches by observing the schema output

Question 1b:

Explanation:

A Boolean field “`is_malformed`” was included to highlight any malformed logs. Then query `writeStreams` were used to display clean and malformed logs to the console.

Screenshot showing malformed handling:

```
-----
Batch: 4
-----
+-----+-----+-----+
|timestamp|level|host|message|
+-----+-----+-----+
|2008-11-09 20:41:32|INFO|10.251.43.115|BLOCK* NameSystem.addStoredBlock: blockMap updated: 10.251.43.115:50010 is added to blk_30509205874280 79149 size 67108864|false|
+-----+-----+-----+

Batch: 5
-----
+-----+-----+-----+
|timestamp|level|host|message|
+-----+-----+-----+
|2008-11-09 20:43:24|INFO|10.251.203.80|BLOCK* NameSystem.addStoredBlock: blockMap updated: 10.251.203.80:50010 is added to blk_78889463318047 32825 size 67108864|false|
+-----+-----+-----+

Batch: 6
-----
+-----+-----+-----+
|timestamp|level|host|message|
+-----+-----+-----+
|2008-11-09 20:44:53|INFO|10.250.11.85|BLOCK* NameSystem.addStoredBlock: blockMap updated: 10.250.11.85:50010 is added to blk_2377150260128098 806 size 67108864|false|
+-----+-----+-----+
```

Updated Schema Screenshot:

```
2026-01-08 17:26:44,839 INFO handler.ContextHandler: Started o.s.j.s.ServletContextHandler@34d0b0bc{/static/sql,null,AVAILABLE,@Spark}
root
|-- value: string (nullable = true)
|-- timestamp: timestamp (nullable = true)
|-- level: string (nullable = true)
|-- component: string (nullable = true)
|-- host: string (nullable = true)
|-- message: string (nullable = true)
|-- is_malformed: boolean (nullable = false)
```

Malformed record policy:

Any log line that was missing a timestamp or level was identified as malformed. To do this, a Boolean column, `is_malformed` was used to flag such cases. These logs were kept in the DataFrame for monitoring and analysis; however are separated to ensure they don't cause errors to the streaming job.

Question 2:

Explanation:

The filter operation was used to ensure the watermark works on clean logs only. Then, the `withWatermark("timestamp", "5 seconds")` was used to demonstrate that events $\leq$ 5 seconds were processed and events $>$ 5 seconds were discarded. The `groupBy` and

window operations were used for event-time aggregation to make the watermark visible.

Streaming code snippet applying Watermark:

```
#applying watermark
watermark_df = (
    parsed_df
    .filter(col("is_malformed")==False)
    .withWatermark("timestamp", "5 seconds")
    .groupBy(window(col("timestamp"), "10 seconds"), "level")
    .count()
)

#console output
query = (
    watermark_df
    .writeStream
    .format("console")
    .outputMode("update")
    .option("truncate", "false")
    .start()
)
```

Micro-batch where late event is included:

Batch: 59

window	level	count
[2008-11-10 10:30:40, 2008-11-10 10:30:50]	INFO	8
[2008-11-10 10:30:50, 2008-11-10 10:31:00]	INFO	5

Batch: 60

window	level	count
[2008-11-10 10:31:00, 2008-11-10 10:31:10]	INFO	2
[2008-11-10 10:31:10, 2008-11-10 10:31:20]	INFO	2
[2008-11-10 10:30:50, 2008-11-10 10:31:00]	INFO	8

Micro-batch where late event is discarded:

Batch: 61

window	level	count
[2008-11-10 10:31:10, 2008-11-10 10:31:20]	INFO	8

Batch: 62

window	level	count
[2008-11-10 10:31:20, 2008-11-10 10:31:30]	INFO	2
[2008-11-10 10:31:30, 2008-11-10 10:31:40]	INFO	3
[2008-11-10 10:31:10, 2008-11-10 10:31:20]	INFO	10

Explanation:

In batch 59, the time window [2008-11-10 10:30:50, 2008-11-10 10:31:00] has an initial count of 5. However, in batch 60 the same time window has a count of 8. This suggests 3 events were late, these events timestamps belonged to this window and they arrived within the 5-second watermark, thus these late events were correctly processed.

In batch 61, we have a window of [2008-11-10 10:31:10, 2008-11-10 10:31:20], thus the watermark will be set to 10:31:15. Whereas, in batch 62 any log record with a timestamp of 10:30:50 that arrives in batch 62 will be discarded as they arrive later than the 5-second watermark. This is shown by window [2008-11-10 10:30:50, 2008-11-10 10:31:00] with count 8 in batch 60 but does not appear in windows 61 or 62 suggesting Spark to close the window and any further events that arrive in this window to be disregarded and their state was cleaned up

Question 3a:

Query Code with Sliding Window Aggregation:

```
#####
#Sliding Window Analysis
#####

#sliding event time window
sliding_df = (
    parsed_df.filter(col("is_malformed")==False)
    .filter(col("component").contains("DataNode"))
    .groupBy(window(col("timestamp"), "60 seconds", "30 seconds"))
    .count()
)

#console output
query = (
    sliding_df
    .writeStream
    .format("console")
    .outputMode("update")
    .option("truncate", "false")
    .start()
)
```

Micro-batch screenshots:

Batch: 4

window	count
[2008-11-09 21:08:00, 2008-11-09 21:09:00]	2
[2008-11-09 21:06:00, 2008-11-09 21:07:00]	2
[2008-11-09 21:04:00, 2008-11-09 21:05:00]	1
[2008-11-09 21:07:30, 2008-11-09 21:08:30]	2
[2008-11-09 21:06:30, 2008-11-09 21:07:30]	3
[2008-11-09 21:04:30, 2008-11-09 21:05:30]	1
[2008-11-09 21:07:00, 2008-11-09 21:08:00]	1

Batch: 5

window	count
[2008-11-09 21:11:30, 2008-11-09 21:12:30]	1
[2008-11-09 21:08:00, 2008-11-09 21:09:00]	3
[2008-11-09 21:13:00, 2008-11-09 21:14:00]	1
[2008-11-09 21:09:30, 2008-11-09 21:10:30]	1
[2008-11-09 21:09:00, 2008-11-09 21:10:00]	2
[2008-11-09 21:10:30, 2008-11-09 21:11:30]	1
[2008-11-09 21:13:30, 2008-11-09 21:14:30]	1
[2008-11-09 21:07:30, 2008-11-09 21:08:30]	3
[2008-11-09 21:08:30, 2008-11-09 21:09:30]	1
[2008-11-09 21:12:00, 2008-11-09 21:13:00]	1
[2008-11-09 21:10:00, 2008-11-09 21:11:00]	1

```
-----
```

Batch: 6	

window	count

[2008-11-09 21:20:00, 2008-11-09 21:21:00]	1
[2008-11-09 21:15:30, 2008-11-09 21:16:30]	1
[2008-11-09 21:18:30, 2008-11-09 21:19:30]	1
[2008-11-09 21:19:30, 2008-11-09 21:20:30]	1
[2008-11-09 21:22:00, 2008-11-09 21:23:00]	2
[2008-11-09 21:14:00, 2008-11-09 21:15:00]	1
[2008-11-09 21:16:00, 2008-11-09 21:17:00]	1
[2008-11-09 21:19:00, 2008-11-09 21:20:00]	1
[2008-11-09 21:14:30, 2008-11-09 21:15:30]	1
[2008-11-09 21:21:30, 2008-11-09 21:22:30]	2

Explanation:

The filter operation was used to filter log event where component includes "DataNode". The window function was used with the groupBy operation to create an event_time sliding window of length 60 seconds and the count() function is used to count the log occurrences per window.

Peak behaviour and Overlap effects:

We have 60 second windows which slide every 30 seconds. This means that each log event will be present in two overlapping windows. Counts in the active window are updated when new DataNodes arrive. Peaks appear when a cluster of DataNodes lie in the overlapping region, thus leading to the event counts for these windows being higher.

Question 3b:

Explanation:

To obtain a full list of valid hosts, the filter function was used to obtain clean logs and remove any instances where the host field was empty. The withWatermark function, with a 60-second window, was used to disregard any old states. The groupBy function was used to aggregate per host and the table was sorted in descending count order using orderBy and desc. In the writeStream output mode, complete mode was used to show the full host table in each micro-batch.

Host-state aggregation code snippet:

```
#Host state aggregation
host_df = (
    parsed_df
    .filter(col("is_malformed")==False)
    .filter(col("host") != "")
    .withWatermark("timestamp", "60 seconds")
    .groupBy("host")
    .count()
    .orderBy(desc("count"))
)

#console output
query = (
    host_df
    .writeStream
    .format("console")
    .outputMode("complete")
    .option("truncate", "false")
    .start()
)
```

Sorted output screenshot:

```

-----
Batch: 14
-----
+-----+-----+
|host      |count|
+-----+-----+
|10.250.7.244|4|
|10.251.201.204|4|
|10.250.19.16|3|
|10.251.71.146|3|
|10.251.126.255|3|
|10.251.31.5|3|
|10.250.5.237|3|
|10.250.6.214|3|
|10.251.111.130|3|
|10.251.42.84|3|
|10.250.6.191|3|
|10.251.110.8|3|
|10.251.35.1|3|
|10.251.75.79|3|
|10.251.75.228|3|
|10.251.73.220|3|
|10.251.202.134|2|
|10.251.25.237|2|
|10.251.67.4|2|
|10.251.67.225|2|
+-----+-----+
only showing top 20 rows

```

State eviction and watermark effect:

The watermark is present to ensure that Spark to remove state for hosts that produce events after the allowed lateness period. This prevents the state from indefinite growth.

Question 4:

Explanation:

The Spark filter function was used to obtain the INFO logs and the messages containing block activity. Aggregation was carried out by groupBy and count, which gave the block count per host. The current_timestamp function was used to include the timestamp for each host in the table. Results were emitted every 15 seconds using .trigger(processingTime="15 seconds").

Query code with trigger setup:

```

#filter logs for messages containing a block ID
#group by host for per-host block activity
block_df = (
    parsed_df
    .filter(col("is_malformed")==False)
    .filter(col("level")==INFO)
    .filter(col("message").contains("blk_"))
    .groupBy("host")
    .count()
    .withColumn("timestamp", current_timestamp())
    .orderBy(desc("count"))
)
#console output
query = (
    block_df
    .writeStream
    .format("console")
    .outputMode("complete")
    .trigger(processingTime="15 seconds")
    .option("truncate", "false")
    .start()
)

```

Batch output screenshot:

Batch: 1

host	count	timestamp
	10	2026-01-06 15:21:10.402
10.251.111.130	2	2026-01-06 15:21:10.402
10.250.11.85	1	2026-01-06 15:21:10.402
10.251.110.8	1	2026-01-06 15:21:10.402
10.251.74.79	1	2026-01-06 15:21:10.402
10.251.123.1	1	2026-01-06 15:21:10.402
10.251.123.132	1	2026-01-06 15:21:10.402
10.251.74.134	1	2026-01-06 15:21:10.402
10.251.214.112	1	2026-01-06 15:21:10.402
10.251.31.180	1	2026-01-06 15:21:10.402
10.251.70.211	1	2026-01-06 15:21:10.402
10.251.43.115	1	2026-01-06 15:21:10.402
10.251.73.220	1	2026-01-06 15:21:10.402
10.251.89.155	1	2026-01-06 15:21:10.402
10.251.75.228	1	2026-01-06 15:21:10.402
10.251.42.84	1	2026-01-06 15:21:10.402
10.251.91.229	1	2026-01-06 15:21:10.402
10.251.38.197	1	2026-01-06 15:21:10.402
10.251.30.6	1	2026-01-06 15:21:10.402
10.251.199.19	1	2026-01-06 15:21:10.402

only showing top 20 rows

Most active hosts:

Logs with an empty host field had the highest activity as their blk_count was the highest. Aside from these, the host with the highest activity is 10.251.111.130 in batch 1.

This suggests at this time period host 10.251.111.130 is carrying out the most block level activity, while the rest only contribute a single event.

Question 5:

Explanation:

A checkpoint directory is configured using `.option("checkpointLocation", f"s3a://{os.environ['BUCKET_NAME']}/checkpoints/task4")` where Spark stores the state in the directory. The query is then stopped using Ctrl+C and restarted with the same checkpoint path. Spark should then restore the state and counts for previously stored hosts should be the same. In the writeStream output mode, complete mode was used to show the full host table in each micro-batch, which makes it easy to compare the original and recovered state.

Checkpoint Recovery code snippet:

```
#group event count per host
df_host = (
    parsed_df
    .filter(col("is_malformed")==False)
    .filter(col("host") != "")
    .groupBy("host")
    .count()
    .orderBy(desc("count"))
)

#console output including checkpoint configuration
query = (
    df_host
    .writeStream
    .format("console")
    .outputMode("complete")
    .option("checkpointLocation", f"s3a://{os.environ['BUCKET_NAME']}/checkpoints/task4")
    .option("truncate", "false")
    .start()
)
```

Batch: 4

host	count
10.251.126.255	7
10.250.7.244	7
10.251.201.204	7
10.250.10.100	6
10.251.214.67	6
10.250.5.237	6
10.250.6.191	6
10.251.73.220	6
10.251.43.115	6
10.251.31.5	5
10.251.70.211	5
10.251.42.84	5
10.251.126.22	5
10.251.203.149	5
10.251.107.19	5
10.251.35.1	5
10.251.67.225	5
10.251.195.33	5
10.251.111.37	5
10.251.74.79	5

only showing top 20 rows

After recovery screenshot:

Batch: 5

host	count
10.251.126.255	7
10.251.201.204	7
10.250.7.244	7
10.250.10.100	6
10.251.214.67	6
10.250.5.237	6
10.250.6.191	6
10.251.43.115	6
10.251.73.220	6
10.251.31.5	5
10.251.126.83	5
10.251.111.37	5
10.251.106.50	5
10.251.70.211	5
10.251.30.85	5
10.251.126.22	5
10.251.195.33	5
10.251.107.19	5
10.251.67.225	5
10.251.203.149	5

only showing top 20 rows

Fault tolerance limitations:

Checkpointing is used for fault tolerance to save the state and progress, which allows a query to restart and recover the state to continue processing from the last committed offsets. However, limitations to checkpoints do exist. Schema changes when re-running can cause complications with the saved state. In addition, corrupted checkpoints can make recovery difficult and thus, the progress may be permanently lost.

Spark APIs used in Task 4:

The Spark DataFrame API was used in this task as DataFrame operations such as filter, regexp_extract and withColumn were used to extract fields from raw logs for parsing and processing data to compute statistics and construct tables. Event-time windowing and watermarking were also crucial to carry out time-based aggregations and efficiently handle late data. The Spark Structured Streaming API was used to ingest the raw log data from a socket source using readStream and then using writeStream the results were output to the console. This allowed the log data to be processed as a continuous stream of data.

Challenges encountered:

One challenge faced in this task was the restriction on using the orderBy in the append output mode, this caused errors to appear as sorting is only permitted in complete mode. Thus, when I sorted event count per host, I was required to change the output mode to complete. Another challenge was handling malformed log entries and entries with empty hosts. This was addressed by ensuring in each question the data that we don't require is filtered out using the filter functions. When obtaining screenshots of the micro-batches for the fault tolerance and recovery question, I was met with an eof error. However, after re-running, I was still able to obtain the screenshots of consecutive batches, showing that the state was correctly restored.

Insight and Knowledge gained:

This coursework helped me develop a better understanding of Spark Structured Streaming's event-time processing and how continuous data is handled. I also learned that watermarks are a very reliable way to handle late data, which stops indefinite growth of memory. In addition, I was able to understand how checkpointing and recovery allow Spark to carry out state recovery and fault tolerance in streaming systems.

How to run the task4.py script in Jhub:

Each question in task 4 needs to be run separately, as each question defines its own independent streaming query. Therefore, only one task can be executed at a time. However, for each question, the initial regex patterns and parsing need to be kept and run for every question. Also, the small part in q1b where the is_malformed field is added should also be kept for q2-q5, as the malformed logs are filtered out in these questions. This then allows each specific question to be commented when it's not being run and then uncommented when it needs to be run for testing. This ensures each question can be run correctly and in isolation.

Spark App ID for task4:

For task 4 there is no Spark App ID as the job history server only shows the app ID for completed jobs. Streaming jobs run indefinitely and so won't appear in the job history server.